Cryptographie sur les courbes elliptiques

Définitions et Opérations Arithmétiques

Thomas BAVOUX, Selma DOUAGI

December 16, 2025

Objectifs de la Présentation

- Comprendre les **fondations mathématiques** de la Cryptographie à Courbe Elliptique (CCE).
- Étudier l'application principale de la CCE pour la **Signature Numérique** (ECDSA).
- Découvrir le protocole de **Chiffrement Hybride** (ECIES) et ses applications pratiques.

Définition d'une Courbe Elliptique

Une courbe elliptique E est définie par l'équation :

$$y^2 = x^3 + ax + b$$

Pour la cryptographie (ECDSA), nous travaillons sur un **Corps Fini** $\mathbb{F}_p$ où p est un nombre premier. L'équation devient :

$$y^2 \equiv x^3 + ax + b \pmod{p}$$

- **Les éléments :** L'ensemble des points (x, y) satisfaisant l'équation, plus un point spécial : le **Point à l'Infini** ($\mathcal{O}$).
- **Non-singularité :** Pour assurer une courbe lisse (pas de points de rebroussement), le discriminant doit être non nul : $\Delta = 4a^3 + 27b^2 \not\equiv 0 \pmod{p}$.

L'Opération d'Addition $(P + Q)$

L'addition de deux points P et Q sur la courbe donne un troisième point R.

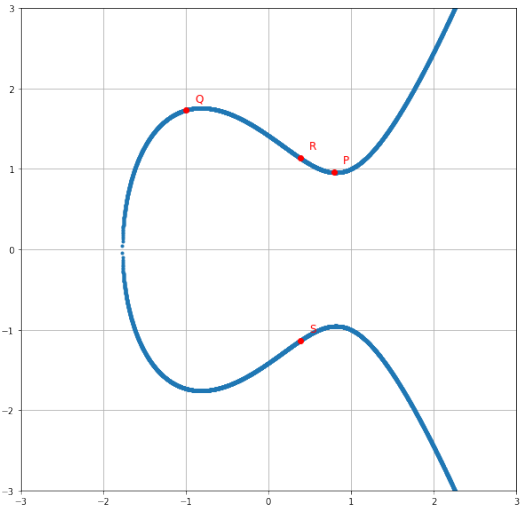
Géométriquement :

1. Tracer la droite passant par P et Q.
2. Elle coupe la courbe en un 3ème point R'.
3. Le résultat R est le symétrique de R' par rapport à l'axe des abscisses.

Élément neutre :

$$P + \mathcal{O} = P \quad \text{et} \quad P + (-P) = \mathcal{O}$$

où $-P$ est le point $(x, -y)$.



Implémentation dans $\mathbb{F}_p$

Soient $P = (x_P, y_P)$ et $Q = (x_Q, y_Q)$. On calcule d'abord la pente s (slope).

Cas A : Addition ($P \neq Q$)

$$s \equiv \frac{y_Q - y_P}{x_Q - x_P} \pmod{p}$$

Cas B : Doublement ($P = Q$)

$$s \equiv \frac{3x_P^2 + a}{2y_P} \pmod{p}$$

Résultat $R = (x_R, y_R)$:

$$x_R \equiv s^2 - x_P - x_Q \pmod{p}$$

$$y_R \equiv s(x_P - x_R) - y_P \pmod{p}$$

La Multiplication Scalaire ($k \cdot P$)

La multiplication scalaire consiste à additionner un point P avec lui-même k fois :

$$Q = k \cdot P = \underbrace{P + P + \cdots + P}_{k \text{ fois}}$$

- **Facile à calculer :** Grâce à l'algorithme "Double-and-Add" (complexité logarithmique).
- **Difficile à inverser :** Retrouver k à partir de Q et P est le problème du **Logarithme Discret (ECDLP)**. C'est ce qui sécurise votre clé privée.

Exemple du notebook ($p = 41$)

Clé privée $d = 17$, Point $G = (12, 5)$. Clé publique $Q = 17 \cdot G = (17, 17)$.

L'Ordre de la Courbe et du Point

Ordre d'un point G

L'ordre n d'un point G est le plus petit entier positif tel que :

$$n \cdot G = \mathcal{O} \quad \text{(Point à l'infini)}$$

- Cela signifie qu'après n additions, on "boucle" et on revient à l'élément neutre.
- **Théorème de Hasse :** L'ordre N de la courbe (nombre total de points) est proche de p :

$$|N - (p+1)| \leq 2\sqrt{p}$$

Application : Dans le notebook, pour $p = 41$, l'ordre du point générateur G trouvé est $n = 21$.

Résumé des Opérations

L'implémentation manuelle de l'ECDSA repose entièrement sur ces briques :

1. **Corps Fini :** Calculs modulo p pour limiter la taille des nombres.
2. **Addition :** Combinaison géométrique traduite en formules algébriques.
3. **Multiplication :** Série optimisée d'additions et de doublements.
4. **Ordre :** Définit la taille de l'espace des clés (sécurité).

Ces opérations mathématiques pures permettent de construire les primitives de *Signature* et de *Vérification*.

Elliptic Curve Digital Signature Algorithm

- L'ECDSA est un **schéma de signature numérique** basé sur la Cryptographie à Courbe Elliptique (CCE).
- C'est un standard essentiel pour l'authentification et l'intégrité des données (e.g., cryptomonnaies, protocoles TLS/SSL).
- Sa sécurité repose sur la difficulté du **Problème du Logarithme Discret sur Courbe Elliptique (ECDLP)**.

Avantage Majeur : Efficacité

Pour un niveau de sécurité équivalent, l'ECDSA utilise des clés **beaucoup plus courtes**.

- Ex: Clé ECDSA de 256 bits $\approx$ Clé RSA de 3072 bits.

$\rightarrow$ Signatures réduites, calculs rapides, moindre consommation de ressources.

Le Problème du Logarithme Discret sur Courbe Elliptique

- L'ECDLP repose sur une **fonction à sens unique** créée par l'opération de multiplication scalaire sur la courbe.
- Cette asymétrie garantit que la **clé privée** (l'entier k) reste secrète, même si la clé publique (P) est connue de tous.

Le Sens Facile (Multiplication)

Objectif : Calculer le point P.

- **Entrée :** Clé Privée k (entier) et Point de Base G.
- **Opération :** $P = k \times G$
- **Résultat :** Clé Publique P (point).

$\rightarrow$ Très rapide, facile à exécuter (quelques millisecondes).

Le Sens Difficile (Logarithme)

Objectif : Trouver la clé privée k.

- **Entrée :** Clé Publique P et Point de Base G.
- **Opération :** Trouver k tel que $P = kG$.
- **Résultat :** Clé Privée k.

$\rightarrow$ Extrêmement lent, impossible à exécuter en temps utile (nécessite un temps exponentiel).

Protocole à Clé Publique

L'ECDSA est structuré autour de trois opérations principales:

1. **Génération des clés :**
   - Une courbe elliptique et un point générateur G sont choisis.
   - Une **clé privée** secrète d est sélectionnée.
   - La **clé publique** Q est calculée par $Q = d \cdot G$ (multiplication scalaire).
2. **Signature :**
   - La clé privée d est utilisée pour générer une signature unique (r, s) pour le message.
3. **Vérification :**
   - La clé publique Q est utilisée pour vérifier l'authenticité de la signature (r, s) et l'intégrité du message.

Génération de la Paire de Clés (Privée/Publique)

Chaque utilisateur génère sa propre paire de clés à partir des paramètres de domaine.

- **Clé Privée k (Le Secret) :** Un **entier aléatoire** de grande taille, choisi dans l'intervalle $[1, n-1]$, avec n l'ordre du point générateur G. C'est un immense nombre premier.
- **Clé Publique P (Le Partage) :** Le point P résultant de la multiplication scalaire :

$$P = k \times G$$

$\rightarrow$ La clé publique P est calculée à partir de k, mais **l'inverse est impossible** à cause de l'ECDLP.

Algorithme de signature

Contexte et Paramètres

- **Paramètres Publics (connus) :** Courbe E, Point générateur G, Ordre n.
- **Clé Privée du Signataire :** k_{priv} (entier secret, $1 < k_{priv} < n$).
- **Message :** M à signer.

1. **Hachage du Message**
   - Calculer $h = HASH(M)$, où h est interprété comme un entier.
2. **Génération du Nonce Aléatoire**
   - Choisir un entier aléatoire et secret r (le nonce), avec $1 < r < n$.
3. **Calcul du Premier Composant (R)**
   - Calculer le point éphémère $R = r \times G$.
   - Extraire sa coordonnée x_R et calculer $R_{sig} = x_R \pmod{n}$.
   - **Note :** Si $R_{sig} = 0$, reprendre l'étape 2.
4. **Calcul du Second Composant (S)**
   - Calculer $S = r^{-1} \times (h + R_{sig} \times k_{priv}) \pmod{n}$.
   - **Note :** r^{-1} est l'inverse modulaire de r modulo n. Si $S = 0$, reprendre l'étape 2.

Résultat de la signature

Résultat

La **Signature** est le couple (R_{sig}, S). Elle est envoyée au vérificateur avec le message M.

Vérifier l'Authenticité et l'Intégrité d'une Signature (R_{sig}, S)

Contexte et Données Reçues par le Vérificateur

- **Paramètres Publics :** Courbe E, Point générateur G, Ordre n.
- **Clé Publique du Signataire :** P (point sur la courbe).
- **Données Reçues :** Message M et Signature (R_{sig}, S).

1. **Vérification Initiale**
   - Vérifier que R_{sig} et S sont des entiers dans l'intervalle $[1, n-1]$. Si non, la signature est invalide.
2. **Calcul du Hachage et de l'Inverse**
   - Calculer $h' = HASH(M)$ (le hachage local du message reçu).
   - Calculer $w = S^{-1} \pmod{n}$ (l'inverse modulaire de S).
3. **Calcul des Facteurs de Vérification (u_1, u_2)**
   - $u_1 = h' \times w \pmod{n}$
   - $u_2 = R_{sig} \times w \pmod{n}$
4. **Reconstitution du Point (R')**
   - Calculer le point de vérification $R' = u_1 \times G + u_2 \times P$.

Résultat de la vérification

Conclusion Finale

- Extraire la coordonnée $x_{R'}$ du point R' et calculer $V = x_{R'} \pmod{n}$.
- **La signature est valide SI et seulement SI $V = R_{sig}$.**

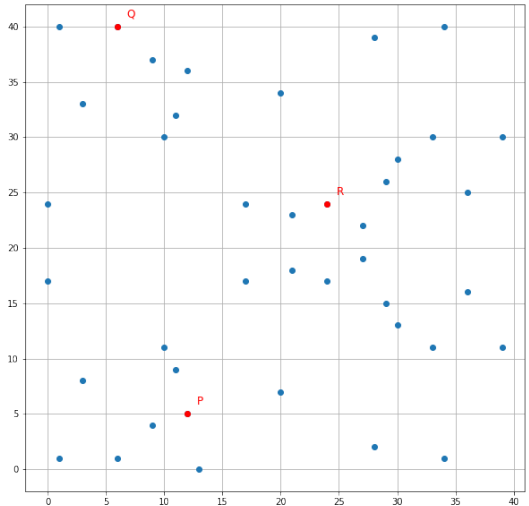
ECDSA avec un petit corps fini : $p = 41, a = -2, b = 2$

- Corps fini : $p = 41$
- Paramètres : $a = -2, b = 2$
- Point générateur G choisi : $(12, 5)$
- Ordre n du point G : $n = 21$
- Clé privée d (choisie aléatoirement) : $d = 17$
- Clé publique $Q = d \cdot G$: $Q = (17, 17)$

Test d'Addition

$P_f = (12, 5),\ Q_f = (6, 40)$

$$P_f + Q_f = (24, 24)$$



Simulation d'Attaque : Force Brute

Le but de l'attaque est de trouver la clé privée d à partir de la clé publique $Q = d \cdot G$.

- On connaît : Clé publique Q, paramètres a, b, p, et le point générateur G.
- On teste toutes les clés privées candidates $d_{cand} \in [1, n-1]$.
- On vérifie si $d_{cand} \cdot G = Q$.

Simulation avec $n = 19792$

Courbe plus complexe :
$p = 78791, n = 19792$

- Clé privée (vraie) : **11743**

Brute force avec n = 19792

Clé privée trouvée : 11743 en 3.7 secondes

Conclusion de la Simulation

L'attaque est **rapide** car le corps fini $p = 78791$ est minuscule. **Dans la réalité**, les courbes utilisent $n \approx 2^{256}$.

Le danger des implémentations manuelles

Le Danger du Nonce r

La signature ECDSA dépend du nonce r (nombre aléatoire secret, $r \in [1, n-1]$).

- **Une réutilisation, une prévisibilité, ou une fuite partielle de r** permet à un attaquant de **calculer la clé privée d en quelques secondes**.
- L'implémentation "à la main" est à proscrire : **elle est uniquement à but pédagogique**.

La Seule Approche Sécurisée

Utiliser des librairies cryptographiques officielles et auditées (comme OpenSSL, Bouncy Castle, modules Python/Go).

- Elles intègrent des contre-mesures strictes (attaques par canal auxiliaire).
- Elles garantissent la qualité cryptographique du nonce r.

Principales Courbes Utilisées pour la Signature et l'Échange de Clés

Nom Courbe	Type	Taille (bits)	Usage Principal
NIST P-256	Courbe Prime (`secp256r1`)	256	Standard Web (TLS/SSL, Certificats)
secp256k1	Courbe Koblitz	256	Cryptomonnaies (Bitcoin, Ethereum)
Ed25519	Courbe Edwards Torsadée	255	Protocoles Modernes (SSH, Signal)

Le Choix Clé : Efficacité

Toutes ces courbes offrent un niveau de sécurité équivalent à une clé RSA de 3072 bits, mais avec des clés 12 fois plus petites et des calculs plus rapides.

Ressource Complémentaire

Pour obtenir plus de détails sur les paramètres de ces courbes :

Normes des Courbes Elliptiques : `https://std.neuromancer.sk/`

Elliptic Curve Integrated Encryption Scheme (ECIES)

- ECIES est un **protocole de chiffrement hybride** qui combine la cryptographie à clé publique (ECC) et le chiffrement symétrique.
- Il garantit la **confidentialité** et l'**authenticité** des messages.
- **Principe :** Alice chiffre un message en utilisant uniquement la clé publique de Bob (Q_{Bob}), en générant une clé de session unique et temporaire.

Avantage Majeur

Il permet un chiffrement unidirectionnel sécurisé : seul le destinataire légitime peut déchiffrer, même si l'échange a lieu sur un canal public (comme une blockchain).

Les Trois Piliers du Protocole Hybride

ECIES est un schéma intégré basé sur l'assemblage de briques cryptographiques :

1. **ECDH (Elliptic Curve Diffie-Hellman)**
   - **Rôle :** Utiliser la multiplication scalaire pour créer un **Secret Partagé** S unique et éphémère entre l'expéditeur et le destinataire.
2. **KDF (Key Derivation Function)**
   - **Rôle :** Dériver une clé de chiffrement symétrique (K_E) et de vérification (K_M) de haute qualité à partir du secret S. (Utilisation courante : HKDF-SHA256).
3. **Chiffrement Symétrique Authentifié (AES-GCM)**
   - **Rôle :** Chiffrer rapidement le message (Ciphertext). Le mode GCM ajoute un **Tag** qui garantit l'**intégrité** : si une seule donnée est modifiée, le déchiffrement échoue.

De l'idée au Ciphertext

1. **Clé Éphémère :** Alice sélectionne une clé privée r aléatoire et calcule le point public éphémère $R = r \cdot G$.
2. **Secret Partagé :** Alice calcule le secret $S = r \cdot Q_{Bob}$ (où Q_{Bob} est la clé publique de Bob).
3. **Dérivation :** Elle utilise S pour dériver la clé symétrique K_E.
4. **Chiffrement :** Elle chiffre le message avec K_E (via AES-GCM).
5. **Transmission :** Alice envoie à Bob l'ensemble du paquet : $(R, IV, Ciphertext, Tag)$.

La Preuve par Charlie - Mise en place

Scénario de Test

Alice chiffre un message pour Bob en utilisant ECIES. Charlie (l'espion) intercepte le paquet $(R, IV, Ciphertext, Tag)$ et tente de le déchiffrer avec sa propre clé privée.

Tentative de lecture :

- **Bob (Détenteur légitime) :** Bob utilise sa clé privée statique d_{Bob} pour calculer le secret partagé $S' = d_{Bob} \cdot R$. Le déchiffrement est possible.
- **Charlie (Attaquant) :** Charlie tente de calculer un secret $S'' = d_{Charlie} \cdot R$. S'' est mathématiquement différent de S'. La clé K''_E dérivée par Charlie est fausse. Le déchiffrement AES-GCM échoue.

Cas d'Usage : ECIES pour la Sécurité des Fichiers

Contexte : Protection d'un Rapport Médical

ECIES est idéalement utilisé pour sécuriser l'échange ou le stockage de documents volumineux et sensibles (ici, un PDF).

Processus de Chiffrement

- **Destinataire :** Bob (Clé Publique `...960e877`)
- **Fichier source :** `cours_ti.pdf`
- **Résultat :** Fichier chiffré `document.enc` (illisible)

Opération clé

Le protocole ECIES utilise le point de la courbe comme clé secrète jetable pour encapsuler la clé symétrique (AES) qui, elle, va chiffrer l'intégralité du PDF.

Conclusion de l'Expérience de Chiffrement de Fichiers

- **Succès du déchiffrement :** Bob réussit à récupérer `document_decrypt.pdf` sans altération, prouvant la fiabilité du processus hybride.
- **Sécurité Garantie :** Si le fichier `document.enc` est intercepté, sans la clé privée de Bob (`...75d2f0d6`), le fichier reste crypté et inutilisable.
- **Intégrité :** Le mode GCM de l'AES garantit que le document n'a pas été corrompu ni modifié pendant le transport ou le stockage.

Validation Finale

L'expérience confirme la robustesse d'ECIES pour la gestion de la confidentialité et de l'intégrité des données à grande échelle (chiffrement de fichiers volumineux).

Synthèse des Concepts

- La CCE est la base de la cryptographie moderne, offrant une **grande sécurité** avec des **clés courtes** (ECDSA).
- L'ECDLP est le fondement mathématique qui protège les clés privées.
- Le protocole ECIES étend la CCE au chiffrement, combinant **ECDH** pour l'échange de clés et **AES-GCM** pour l'intégrité et la confidentialité.

Message Clé

L'implémentation de ces algorithmes doit impérativement utiliser des librairies cryptographiques **officielles et auditées** pour garantir la sécurité et éviter les vulnérabilités liées au nonce ou aux attaques par canal auxiliaire.